

第 4 章处理器调度

调度调频·负载均衡·能效调度·可扩展调度器类

江西科技师范大学

万博洋

2026 年 9 月 24 日 (第 4 教学周)

版权与来源说明

使用声明

本课件仅供教学使用，以《智能终端操作系统与 OpenHarmony 实践》（陈海波等，电子工业出版社）为基准编写。

- 教材插图：按图号标注；第 4 章原文插图有限，已用官方与自制图补足
- 网图：ARM 指南、OpenHarmony 调度文档，图注均标来源
- 课程自制图标注“课程自制”；代码为示意代码，非鸿蒙源码
- 实验输出为模拟器实拍（API 24），命令逐条可复核

本讲目录

- **4.0 调度的对象与目标：进程、线程与调度指标**
- **4.1 平衡性能、功耗、热的调度调频机制**（DVFS 调频 / QoS 服务质量）
- **4.2 负载均衡和公平调度**（PELT 负载追踪 / CFS 完全公平调度）
- **4.3 能耗感知调度（EAS）**
- **4.4 基于可扩展调度器类的调度策略定制（sched_ext）**

每个知识点配一个终端实验：在模拟器上把调度器的决策“看见”。

本讲导学

导学

玩手游久了手机发烫、帧率下降——是谁在决定 CPU 跑多快、任务跑哪个核？本讲从“调度对象”讲起，逐步深入终端特有的调度调频机制：性能、功耗、热三者如何平衡，负载如何均衡，能耗如何感知，策略如何定制。

- 关键词：调度调频三角·DVFS（动态电压频率调节）·QoS（服务质量）·PELT（调度实体粒度负载追踪）·CFS（完全公平调度器）·EAS（能耗感知调度）·sched_ext（可扩展调度器类）
- 带着问题听课：为什么不能在负载高时才临时把频率拉满？

上节回顾

- 第 3 讲：内核形态（宏/微/混合）与鸿蒙通用微内核——内核只提供机制
- 机制中最核心的一条：CPU 调度——决定“谁、在哪个核、以什么频率运行”
- 终端场景的调度比服务器多两个约束：功耗预算与发热红线

第 1 部分

4.0 调度的对象与目标：进程、线程与调度指标

4.0 定义：处理器调度

定义

处理器调度（CPU Scheduling）是内核在多个**就绪任务**（已可运行、排队等 CPU）之间分配 CPU 时间的机制：决定**谁运行、何时运行、运行多久、在哪个核上运行**。它是内核“机制与服务分离”中最基础的一条机制。

- 调度的两个层级：**选核**（任务放到哪个 CPU 核心）与**选任务**（核心本地队列中下一个运行谁）
- 调度的结果直接影响用户体验：响应快不快、帧率稳不稳、手机烫不烫

4.0 调度对象：进程与线程

- **进程**：资源分配的基本单位（地址空间、文件、内存），开销大
- **线程**：CPU 调度的基本单位——同一进程的线程共享地址空间，切换更轻
- 内核视角：每个线程对应一个**任务 (task)**，调度器操作的是“任务”
- Linux 用 `task_struct` 描述任务，其中的调度实体 `sched_entity` 维护调度信息（vruntime 等，4.2 详述）——**调度实体是调度器管理的最小对象**（单个任务或一组任务）

一句话：进程是“户”，线程是“人”；CPU 分的是人，不是户。

4.0 调度时机与五大指标

何时触发调度？

- 任务阻塞（等 I/O、等锁）主动让出；任务被唤醒抢占当前任务；时间片用完；更高优先级任务就绪——每次切换都要**保存/恢复任务现场**（即上下文切换，4.4 实验可实测）

拿什么衡量调度好不好？

- **响应时间**：从就绪到首次运行（交互体验）
- **周转时间**：从提交到完成（批处理）
- **吞吐量**：单位时间完成的任务数（服务器）
- **公平性**：各任务按权重（分配份额，4.2 由 nice 值决定）获得 CPU 时间（多任务共存）
- **时延可预期**：端到端延迟稳定（终端场景，防掉帧）

终端场景把“响应/可预期”放在首位：用户对瞬间卡顿零容忍，对续航差的容忍度次之。

4.0 经典调度算法速览

表: 五类经典调度算法及其终端适用性

算法	思想	优点 / 缺点	终端适用
FCFS	先来先服务, 非抢占	简单 / 短任务被长任务堵住	不适用
短作业优先	最短任务先跑	平均周转最优 / 长任务饿死	不适用
时间片轮转	轮流运行固定时间片	响应公平 / 时间片大小难定	思想基础
优先级调度	高优先级先跑	及时响应 / 低优先级饿死	核心 (见 4.1)
多级反馈队列	队列分级 + 动态升降	兼顾响应与吞吐 / 参数复杂	思想基础

现代调度器不是单一算法, 而是“公平 + 优先级 + 能耗”的组合策略 (4.2/4.3/4.4 逐一展开)。

实验 4.0：看见 CPU 与调度器

先在 PC 终端连接三步，再进 hdc shell 执行

- 连接: `hdc list targets` (空则启动模拟器) $\rightarrow$ `hdc tconn 127.0.0.1:5555` $\rightarrow$ `hdc shell`
- `taskset -p $$`: CPU 亲和掩码 (该任务可用的核心集合), `f` (二进制 1111) = 4 核在线
- `uptime`: 1/5/15 分钟平均负载 ($\div$ 核数 > 1 即排队)
- `ps -ef | wc -l`: 任务总数 (模拟器约 180+)
- `ps -ef | grep '\[' | wc -l`: 内核线程数 (方括号开头, 约 80 个)

观察点: ① 掩码数核 ② 负载对比核数判忙闲 ③ 近 200 任务 + 80 内核线程, 调度器持续决策。

第 2 部分

4.1 平衡性能、功耗、热的调度调频机制

4.1 定义：性能、功耗、热三角

定义

调度调频（Scheduling & Frequency Scaling）指内核在**选核、选任务**（调度）的同时决定**每个核心以什么频率/电压运行**（调频）。终端场景中二者必须联合决策，因为性能、功耗、发热三者互相牵制，构成“三角平衡”。

- **DVFS**（Dynamic Voltage and Frequency Scaling，动态电压频率调节）：按负载动态调频率与电压的机制——本节的调频基础
- 平衡的总原则：**热 > 性能 > 功耗**（教材给出；热是安全红线）

4.1 三大矛盾：为什么不能拉满

- **性能要求越来越高**：120fps 下每帧只有 8.3ms，绘帧超时即丢帧；
240/360Hz 与多窗口进一步加压
- **功耗预算越来越紧**：手机越做越薄，电池可用能量受限，需要更精细的功耗感知
- **发热是强制红线**：国标 GB 4943.1—2022 要求设备表面温度不超过 48°C——超了不是“体验差”，是**不合规**

平衡的基本逻辑

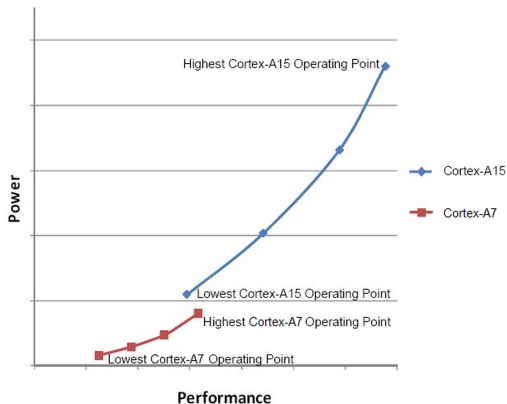
性能是“短期敏感”指标（瞬间卡顿用户立刻感知）；功耗是“长期敏感”指标（续航要几小时才感知）；热是“硬约束”。所以在**满足性能、用户不感知卡顿**的前提下，功耗要最小；温度到线必须降温，且优先牺牲“用户感知最低”的性能。

4.1 传统调度器的三个缺陷

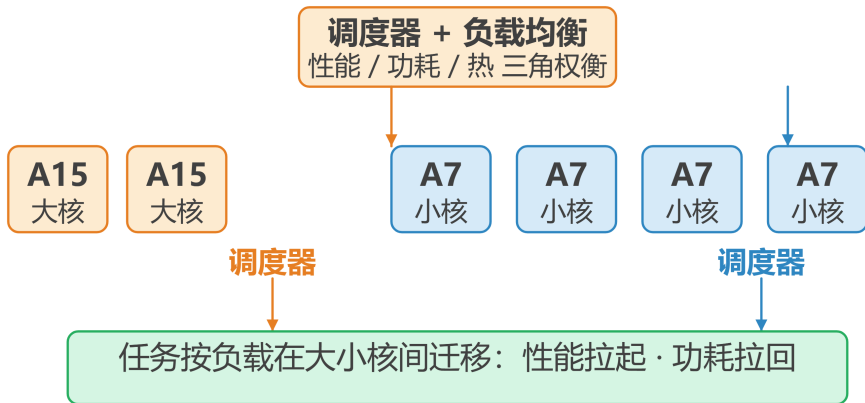
- **① 语义鸿沟：**任务优先级/选核只看“历史运行时长”加权，与用户真实需求脱节——
 - 供给过度：dex2oat 编译负载重，传统调度器往大核高频塞，手机发烫卡顿（其实慢一点跑用户根本无感）
 - 供给不足：绘帧任务只跑几毫秒，被认为是“轻负载”，但每帧只有 8ms 预算——算力给少就掉帧
- **② 缺少统一 QoS（服务质量）支持：**每帧内容不同负载不同，一组静态参数不可能同时保证“小帧不浪费、大帧不掉链”
- **③ 调频各自为政：**CPU/GPU/DDR 各调各的频率，互不感知，“布朗运动式”调整得不到最优资源配比

结论：需要 DDL（Deadline，截止期）感知的算力分配与资源统一决策——4.1 的 QoS 接口与 4.4 的截止期调度正是对此的回答。

4.1 DVFS: 频率-功耗非线性曲线



4.1 大小核异构：性能与功耗的取舍



4.1 现役实例：麒麟 9030 Pro 的三丛集

① 超大核



② 大核



③ 小核



三丛集 · 9 核 14 线程 · 超大核与 4 颗大核支持超线程

三档算力/功耗特性 → 调度器按性能域选核 (4.3 EAS 的基础)

丛集：按算力/功耗特性划分的核心分组 | SMT×2：超线程，一个物理核跑两个线程

首发：Mate 80 Pro (2025-11) | 核参数来源：极客湾实测、GSMArena 等评测 (华为官方未公布)

4.1 现役实例：麒麟 9050 Pro 的四丛集

① 超大核



② 大核



③ 能效核



④ 小核



四丛集·9核16线程·新增「能效核」过渡档位

负载不必在大小核间跳变，选核更平滑——性能域更多、能耗模型更复杂

丛集：按算力/功耗特性划分的核心分组 | SMT×2：超线程，一个物理核跑两个线程

首发：Mate XT 2 (2026-09) | 核参数来源：快科技、新浪科技、百度百科（华为官方未公布）

4.1 QoS：把“用户想要什么”告诉内核

- **QoS** (Quality of Service, 服务质量)：应用通过接口告诉内核“我这个任务的紧急程度”，内核据此分配运行时间与次序
- 开发者比内核更懂应用：把任务按**与用户交互的关联程度**分类，交给 QoS 接口
- OpenHarmony 定义 **6 个 QoS 等级**（枚举名可直接查 `qos.h`），并配套设置接口：

等级梯度（关键三级 + 完整枚举见 `qos.h`）

`QOS_BACKGROUND`（后台同步，可推迟） < `QOS_USER_INITIATED`（用户触发，如打开文档） < `QOS_USER_INTERACTIVE`（UI 线程/动画，即时响应）

效果：后台下载自动“让路”并分到更少时间；动效绘制立即执行且分更多时间。

4.1 QoS 接口：三行代码的调度哲学

OpenHarmony QoS 接口（示意，教材代码 4-1/4-2）

```
1 // 声明：设置/重置/查询当前线程的 QoS 等级
2 int OH_QoS_SetThreadQoS(QoS_Level level);
3 int OH_QoS_ResetThreadQoS(void);
4 int OH_QoS_GetThreadQoS(QoS_Level *level);
5
6 // 调用：把“用户操作触发的任务”标成 USER_INITIATED
7 void *worker(void *args) {
8     OH_QoS_SetThreadQoS(QOS_USER_INITIATED);
9     ... // 高 QoS 下的操作，如图片缩放处理
10 }
```

设计要点：调度决策不再只靠内核猜，应用主动“报需求”——语义鸿沟就此弥合。

实验 4.1：看见 CPU 能力与真机差异

在 hdc shell 中执行

- `top -n 1 -b | head -4`: 最后一行 `400%cpu ... 400%idle`——4 核总算力 (每核标称 100%)
- `hidumper -c base`: 系统信息簇——内核版本、设备型号、ABI
- `cat /proc/meminfo | head -3`: 内存总量/可用量
- `cat /sys/devices/system/cpu/...` (频率档位/温度): **模拟器返回 Permission denied**

观察点: ❶ 总算力 = 核数 × 100% ❷ 模拟器是虚拟 CPU, 不暴露频率/温度——这两类接口的真机观察见作业任务 A。

第 3 部分

4.2 负载均衡和公平调度

4.2 定义：负载均衡与公平调度

定义

负载均衡：监控各 CPU 核心负载，把任务从忙核迁移到闲核，充分利用多核算力；**公平调度**：在同一核心上按权重公平分配 CPU 时间，让每个任务获得与其权重成比例的服务。

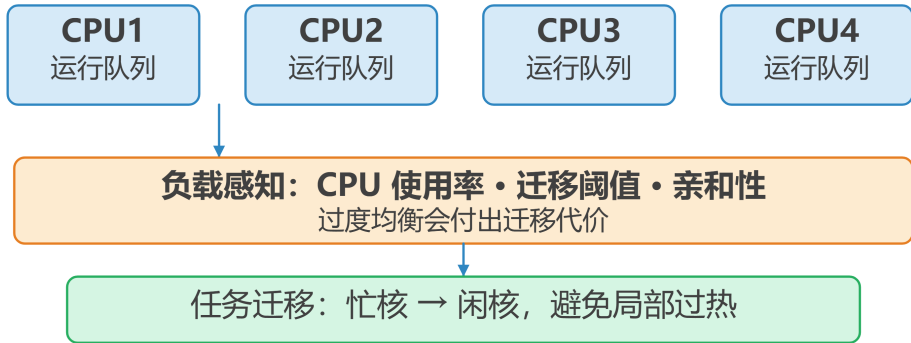
- 两者一个管“横向”（核间均衡），一个管“纵向”（核内公平），共同构成多核调度的地基

4.2 两级调度：先选核，再选任务

- **第一级：选核**——任务创建/唤醒时确定放到哪个 CPU 核心；此后尽量固定在本队列，避免频繁跨核切换
- **第二级：选任务**——在核心的本地运行队列中选下一个运行谁
- 固定带来的问题：**负载不均**——有的核 100% 忙、有的核接近 0%
- 解决：**负载均衡机制**按需迁移任务，把忙核的任务搬到闲核

迁移不是免费的：缓存失效（含地址翻译缓存，第 5 讲详述）、迁移本身有开销——均衡要“恰到好处”。

4.2 负载均衡：从忙核到闲核



4.2 PELT：任务级负载追踪

- 难点：负载动态变化，追踪要又准又省——不准误迁移、太勤拖累系统
- 旧做法（Linux 3.8 前）：以运行队列为粒度——队列越长负载越高，但不知该迁移谁
- **PELT**（调度实体粒度负载追踪）：以**单个任务**为粒度
- 机制：以 1024 微秒为一周期，记录可运行时间 x ；周期负载
 $L = \text{scale_cpu_capacity} \times x/1024$ （该核相对最强核算力， $0 \sim 1024$ ）
- 累计负载用**衰减系数** y 加权： $L = L_0 + L_1y + L_2y^2 + \dots$ （越近贡献越大； $y = 2^{-1/32}$ ，约每 33 ms 减半）；更新只需 $L' = Ly + L_T$ ，**不保存历史**

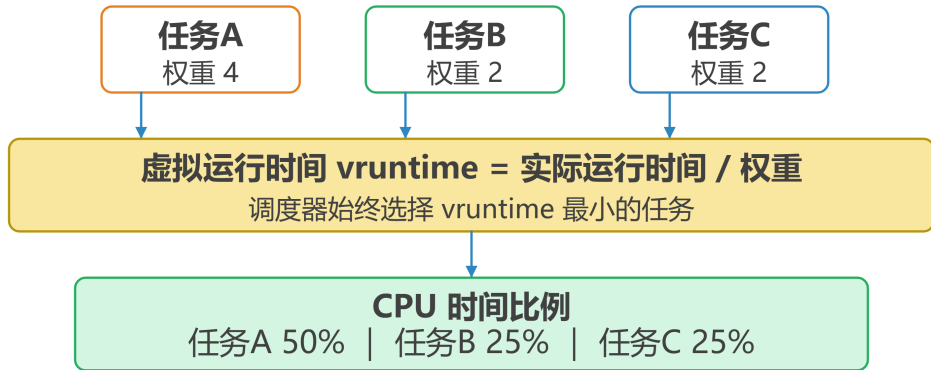
价值：任务与核心都有准确的负载画像，迁移决策有据可依。

4.2 公平调度 CFS：虚拟运行时间

- **CFS** (完全公平调度器)：Linux 2.6.23 起的默认调度器，追求公平分时
- 核心量：**虚拟运行时间 vruntime** = 实际运行时间 ÷ 权重
- 权重由 **nice 值** ($-20 \sim 19$) 换算：优先级越高，vruntime 增长越慢
- 调度规则：**总选 vruntime 最小者**——“谁亏得多谁先跑”
- 数据结构：**红黑树** (自平衡 BST)，**最左节点**即 vruntime 最小者

一句话：负载均衡定“谁来跑”，CFS 定“谁先跑、跑多久”。

4.2 CFS: 权重决定时间比例



实验 4.2：看见负载与优先级

在 hdc shell 中执行

- `top -n 1 -b | head -6`: 各任务 CPU 占用——调度分配的结果
- `ps -e -o pid,ni,comm | sort -k2 -n | head -4`: nice -20 的任务 (如 rcu_gp/kworker)
- `ps -e -o pid,ni,comm | sort -k2 -n | tail -3`: nice 最高的任务 (内核回收线程 zswapd0 等)
- `nice -n 10 sleep 60 & ps -e -o pid,ni,comm | tail -3`: **同一条内起任务并查看** (分开执行会被回收)

观察点: ❶ nice 即 CFS 权重 ❷ 内核任务同样分高低优先级 ❸ 优先级由 nice 决定。

第 4 部分

4.3 能耗感知调度

4.3 定义：能耗感知调度 EAS

定义

EAS (Energy Aware Scheduling, 能耗感知调度): 在保证性能的前提下, 通过把任务放到“总能耗最低”的 CPU 核心上来降低系统能耗的调度机制——CFS 的能效扩展。

- 背景: 任务放大核还是小核能耗不同——EAS 用能耗模型量化差异
- 容量 (Capacity): 当前频率下的处理能力, 标准化到 $0 \sim 1024$, 与 PELT 负载单位统一
- 功率 (Power): 当前频率下的功率 (mW); **能耗 = 时间 × 功率**
- 性能域 PD 与操作性能点 (OPP): 同一 PD 共享“频率-电压”档位表

4.3 能耗模型：一道可算的选择题

表: 某假设混合处理器架构的能耗模型 (教材表 4-1)

档位	大核 (PD0)		小核 (PD1)	
	容量	功率 (mW)	容量	功率 (mW)
OPP1	384	300	128	80
OPP2	768	900	256	180
OPP3	1024	1800	512	440

档位规则：取能容纳该负载的最低档位。算例：负载 200 跑一个单位时间——

- 大核走 OPP1 (384/300 mW)：能耗 = $200/384 \times 300 \approx 156$
- 小核 OPP1 装不下、升 OPP2 (256/180 mW)： $200/256 \times 180 \approx 141$
- 结论：**选小核更省电**；负载 > 512 小核最高档也装不下，只能上大核

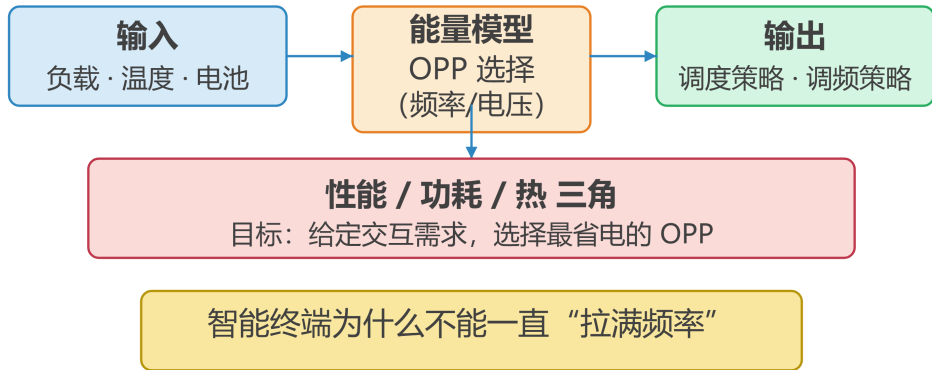
关键前提：功耗随频率**非线性增长** (见图)，高频虽快但更费电——所以“能效最优”不等于“频率最高”。

4.3 EAS 如何做决策

- **① 先保性能**：核心负载超过当前容量时先升频（OPP 上调），满足负载需求
- **② 再算能耗**：观察“剩余容量”（当前容量 - 当前负载）——新任务能塞进剩余容量，就不会触发升频，能耗自然低
- **③ 候选与比较**：每个性能域选“剩余容量最大”的核心作候选，计算迁移后总能耗；**只有迁移后总能耗更低才迁移**
- **④ 适用边界**：任一核心负载超过最高容量 80% 时，EAS 停止参与选核、退回常规负载均衡——高负载先保性能

一句话：高负载先保性能，中低负载才谈省电——与 4.1 的三角原则一脉相承。

4.3 EAS：能量模型驱动的选核决策



实验 4.3：看见省电空间

在 hdc shell 中执行

- `top -n 1 -b | head -4`: 注意空闲率——400%idle 表示 4 核几乎全闲，正是 EAS 谈省电的场景
- `cat /proc/meminfo | head -5`: 内存总量/可用量与缓存
- `hidumper -c system`: 系统内存概况 (Total/Free/Lost RAM 与分类)
- 温度接口 `/sys/class/thermal/*`: 模拟器返回 Permission denied (路径受限)

观察点: ❶ 空闲率高才有降频/省电空间 ❷ 模拟器读不到温度——记录报错原文 ❸ 高负载“保性能”行为要在真机观察。

第 5 部分

4.4 基于可扩展调度器类的调度策略定制

4.4 定义：可扩展调度器类

定义

可扩展调度器类 (Extensible Scheduler Class, sched_ext): 允许在运行时把**用户态编写的调度策略程序**加载进内核、接管调度决策的框架——统一的调度框架 + 可插拔的策略。

- 定位: Linux 主线 6.12 起 (本课模拟器 5.10 内核, 只作原理讲解)
- 动机: 设备形态多样, 内核自带策略难样样最优
- 追问: 用户态代码凭什么能安全进内核? ——靠 **BPF** 机制: 加载时静态分析, 保证不会因越界/死循环拖垮内核 (策略卡住还有 watchdog 兜底)
- 价值: 策略可热插拔, 调度器成为 “可编程平台”

4.4 调度器类：策略插拔示意

示意代码（参考 Linux sched_class 设计），非鸿蒙内核源码

```
1 struct sched_class {                                // 一种调度策略的封装
2     const struct sched_class *next;                 // 串成链表：按优先级排序
3     void (*enqueue)(struct task *p);               // 任务入队
4     void (*dequeue)(struct task *p);               // 任务出队
5     struct task *(*pick_next)(void);               // 选下一个运行的任务
6 };
7 // 统一框架按优先级（链表顺序）依次询问各调度器类
8 for (const struct sched_class *cls = sched_class_highest; cls; cls =
   ↪  cls->next)
9     if ((p = cls->pick_next()))
10         break;
```

4.4 案例：LAVD 给游戏“保帧”

- 场景：游戏卡顿与掉帧最影响体验，优化目标是**平均帧数与最低帧数**
- 实测规律（LAVD 作者分析真实游戏负载）：
 - 任务大多是长驻任务；每次执行仅百微秒级且执行/等待时间稳定——呈现**周期性**
 - 绝大多数调度不是被抢占，而是任务主动等 I/O（epoll、futex_wait：等事件/锁的系统调用）
 - 一个完整工作（如从输入到画面更新）由**一串任务**接力完成——任务链
- **LAVD**（延迟关键性感知虚拟截止期限）调度器：给关键任务更紧的**虚拟截止期限**（按预测执行时长与关键性折算的排序键，非墙钟），优先调度
- 判据：执行越短、唤醒/等待上下游越频繁，危害越大——越“关键”
- 实现路径：LAVD 正是基于 sched_ext 框架用 BPF 编写的调度器

启示：调度目标是端到端延迟可预期，而非单个任务跑了多久。

实验 4.4：看见调度活动

在 hdc shell 中执行

- `cat /proc/1/status | grep ctxt`: init 的上下文切换计数 (开机约 10 分钟: 自愿 1.5k+)
- `cat /proc/1/status | grep Threads`: init 的线程数
- `top -n 1 -b | head -6`: 各任务的 CPU 占用——调度分配的直接结果
- `ps -ef | wc -l`: 全部任务数——调度器持续在这近 200 个任务间切换

观察点: ① ctxt 计数就是调度器的工作量 ② 任务各自持有调度属性 ③ nice 决定权重。

Take-away (本讲小结)

核心要点

- 对象是线程/调度实体；指标：响应、周转、吞吐、公平、可预期
- 调频三角：性能/功耗/热平衡（热 > 性能 > 功耗）；DVFS+QoS 弥合语义鸿沟
- 负载均衡（PELT 追踪）+ 公平调度（CFS：vruntime 红黑树按权重）
- EAS：能耗模型选核；高负载保性能，中低负载省电（80% 阈值）
- sched_ext：策略可插拔；LAVD——按任务链关键性保帧
- 考点：PELT 公式（含参数）、CFS 规则、EAS 算例与 80% 阈值、sched_ext 安全前提

第 4 周实验卡：看见调度

本周必做 (30-60 分钟)

- 连上模拟器，执行实验 4.0~4.4 的命令：核数/负载/线程数/nice/上下文切换
- 挑一个命令，解释它的输出与课本概念的对应关系（如 nice 值 $\leftrightarrow$ CFS 权重）

AI 验证题 (课后 3 题，见作业手册任务 B)

- ① 为什么不能一直把频率拉满？（核对：性能/功耗/热三判据）
- ② PELT 为何用指数衰减？（核对：越近贡献越大）
- ③ EAS 为何过 80% 就退位？（核对：高负载保性能）

第 4 周实验卡：实测命令与输出

```
[1] 4.0 cores | load | tasks
> hdc shell "taskset -p $$"
pid 16847's current affinity mask: f
> hdc shell "uptime"
19:13:01 up 49 min, 0 users, load average: 0.41, 0.29, 0.24
> hdc shell "top -n 1 -b | head -4"
Tasks: 182 total, 1 running, 181 sleeping, 0 stopped, 0 zombie
Mem: 4025356K total, 1878416K used, 2146940K free, 16244K buffers
Swap: 2097148K total, 171520K used, 1925628K free, 682096K cached
400%cpu 0%user 0%nice 0%sys 400%idle 0%iow 0%irq 0%sirq 0%host
[2] 4.2 nice is CFS weight
> hdc shell "ps -e -o pid,ni,comm | sort -k2 -n | head -4"
3 -20 [rcu_gp]
4 -20 [rcu_par_gp]
6 -20 [kworker/0:0H-kblockd]
8 -20 [mm_percpu_wq]
> hdc shell "ps -e -o pid,ni,comm | sort -k2 -n | tail -3"
22568 0 [kworker/u8:1-ext4-rsv-conversion]
70 15 [zswapd0]
80 19 [ion_system_heap]
[3] 4.4 init context switches
> hdc shell "cat /proc/1/status | grep Threads"
Threads: 1
> hdc shell "cat /proc/1/status | grep ctxt"
voluntary_ctxt_switches: 1633
nonvoluntary_ctxt_switches: 2539
```

- ① 亲和掩码数核 (f=4 核) 与负载 ② nice 值分布：高优先级 -20 与低优先级 15/19 ③ init 的上下文切换计数

延伸资源与实践

论文与文档

- **教材第 4 章**：平衡性能/功耗/热的调度调频、PELT、CFS、EAS、sched_ext
- **OpenHarmony 内核调度文档**：<http://docs.openharmony.cn> (调度子系统、QoS 接口)
- **Linux 源码阅读入口**：`kernel/sched/fair.c` (CFS 与 EAS 选核)、`kernel/power/energy_model.c` (能耗模型)

延伸资源与实践（续 1）

实验与预习

- **QoS 动手（计入假期作业加分）：**查 `OH_QoS_SetThreadQoS` 参数表，列出 6 等级适用场景
- **预习：**教材第 5 章“内存管理”——虚拟地址如何变成物理地址、内存不够怎么办

假期作业与安排

时间安排

9 月 24 日课后进入假期自习，**10 月 10 日复课**；假期作业**复课前提交**，见课程主页《第 4 周作业：HarmonySchedLab》。

假期作业四件事（满分 100，手册见课程主页）

- **A•调度观察 30 分**：实验 4.0~4.4 在模拟器执行，截图对应课本概念
- **B•Agent 结对 25 分**：三个调度问题“考”AI，用课件判据核对并留 Prompt
- **C•负载小项目 30 分**：ArkTS 并发压测应用（模板与 AI 提示词见手册）
- **D•阅读思考 10 分**：教材第 4 章两道思考题；另 5 分看日志规范

下次课预告

预告

第 5 讲：内存管理——虚拟地址如何变成物理地址、内存不够怎么办（OOM Killer 与 Hyperhold）

上课时间：**10 月 10 日**（假期后第一课）；假期作业随堂检查